# I am doing a Lab on R, using Positron for one of my labs on Visualizing Information on the nycflights13 dataset.

That's a great lab topic! The `nycflights13` dataset is one of the most popular teaching datasets in R for data visualization. Here's a solid overview to help you get started in **Positron** (Posit's next-gen IDE for R and Python).

### The Dataset

The `nycflights13` package contains **5 related tables** covering all 336,776 flights that departed from NYC airports (EWR, JFK, LGA) in 2013:[1]

- `flights` — on-time data for every departure[2]
- `airlines` — airline name lookup by carrier code
- `airports` — airport metadata (lat/lon, timezone, etc.)
- `planes` — aircraft metadata by tail number
- `weather` — hourly weather data at each NYC airport[1]

### Setup in Positron

Install and load the necessary packages in your R script or notebook:

```
install.packages("nycflights13")  # only once
install.packages("ggplot2")
install.packages("dplyr")

library(nycflights13)
library(ggplot2)
library(dplyr)
```

Positron works seamlessly with these tidyverse packages, so you can run chunks interactively just like in RStudio.[3]

### Common Visualizations

Here are some typical `ggplot2` visualizations for this dataset:[3]

- **Departure delay distribution** — histogram of `dep_delay` to see how often flights are late
- **Delays by carrier** — boxplot of `arr_delay` grouped by `carrier`

- **Flight volume by month** — bar chart using `geom_bar()` on the `month` column
- **Dep delay vs. distance** — scatter plot to explore if longer flights have fewer delays
- **Departure counts by airport** — bar chart faceted or colored by `origin` (EWR, JFK, LGA)

### Quick Example

```
# Histogram of departure delays (excluding extreme outliers)
flights %>%
  filter(dep_delay < 180) %>%
  ggplot(aes(x = dep_delay)) +
  geom_histogram(binwidth = 10, fill = "steelblue", color = "white") +
  labs(title = "Distribution of Departure Delays",
       x = "Departure Delay (minutes)", y = "Count")
```

### Key Variables to Explore

| Variable | Description |
|---|---|
| `dep_delay`, `arr_delay` | Departure/arrival delay in minutes |
| `carrier` | Two-letter airline code |
| `origin`, `dest` | Origin and destination airport codes |
| `month`, `day`, `hour` | Time dimensions for trend analysis |
| `distance` | Flight distance in miles |
| `air_time` | Time in air (minutes) |

The dataset is designed specifically as a teaching resource for data manipulation and visualization in R, so it pairs naturally with `ggplot2` and `dplyr`.[1]

What specific aspect of the lab are you working on — are you trying to build a particular type of chart, wrangle the data with `dplyr`, or something else?

⁂

# For one of the tasks, I would need to rename some of the columns using the replace_values() functions to do the following:

Create a column with the airports' common names. Use the replace_values() function from the dplyr package to convert "EWR" to "Newark" and "LGA" to "LaGuardia"; "JFK" is widely recognized, so you may leave it unchanged.

`replace_values()` is a relatively new `dplyr` function (superseding the older `recode()`) that partially updates an existing vector, keeping unmatched values unchanged — making it perfect for this task.[16] [17]

### How `replace_values()` Works

`replace_values()` retains original values for any cases that don't match, while `recode_values()` (its sibling) would require you to explicitly handle all cases. Since "JFK" should stay unchanged, `replace_values()` is the right choice here — unmatched values are simply kept as-is.[16]

The function supports a **formula-based approach** using `from ~ to` syntax, like `"EWR" ~ "Newark"`, which is clean and readable for a small number of substitutions.[16]

### The Code

Use `mutate()` alongside `replace_values()` to create a new column:

```
library(nycflights13)
library(dplyr)

flights <- flights %>%
  mutate(airport_name = replace_values(origin,
    "EWR" ~ "Newark",
    "LGA" ~ "LaGuardia"
  ))
```

- `mutate()` creates the new column `airport_name` without modifying the original `origin` column[18]
- `"EWR" ~ "Newark"` and `"LGA" ~ "LaGuardia"` are the formula-based replacement rules[16]
- `"JFK"` is not listed, so it remains `"JFK"` automatically since `replace_values()` retains unmatched values[16]

### Verifying the Result

You can quickly check the output with:

```
flights %>%
  distinct(origin, airport_name)
```

This should give you:

| origin | airport_name |
|---|---|
| EWR | Newark |
| LGA | LaGuardia |
| JFK | JFK |

> **Note:** `replace_values()` is available in newer versions of `dplyr`. If you get an error like *"could not find function"*, run `packageVersion("dplyr")` to check your version. If it's below 1.1.0, you may need to update with `install.packages("dplyr")`, or use the older equivalent: `mutate(airport_name = recode(origin, "EWR" = "Newark", "LGA" = "LaGuardia"))`.[17] [19]

# Would this be correct:

departures <- flights |>
filter(!is.na(dep_delay)) |>
mutate(
airline_code = carrier,
airport_code = origin,
airport = replace_values(origin, "EWR" ~ "Newark", "LGA" ~ "LaGuardia"),
on_time = dep_delay < 15
) |>
select(airline_code, airport_code, airport, hour, on_time)

Yes, your code is correct and well-structured! Here's a breakdown of each step:

### Line-by-Line Review

```
departures <- flights |>
  filter(!is.na(dep_delay)) |>
  mutate(
    airline_code = carrier,
    airport_code = origin,
    airport = replace_values(origin, "EWR" ~ "Newark", "LGA" ~ "LaGuardia"),
    on_time = dep_delay < 15
  ) |>
  select(airline_code, airport_code, airport, hour, on_time)
```

| Line | What it does | ✅ Correct? |
|---|---|---|
| `filter(!is.na(dep_delay))` | Removes rows where departure delay is missing | ✅ |
| `airline_code = carrier` | Creates a copy of `carrier` with a descriptive name | ✅ |
| `airport_code = origin` | Creates a copy of `origin` with a descriptive name | ✅ |
| `airport = replace_values(...)` | Maps "EWR"→"Newark", "LGA"→"LaGuardia", "JFK" stays "JFK" | ✅ |
| `on_time = dep_delay < 15` | Creates a logical TRUE/FALSE column | ✅ |
| `select(...)` | Keeps only the 5 relevant columns | ✅ |

### One Thing to Note

The `on_time = dep_delay < 15` logic treats flights with a departure delay of **less than 15 minutes** as on time (including early departures where `dep_delay` is negative). This is a reasonable and common industry-style threshold — just make sure it matches what your lab instructions specify, as some tasks define "on time" as `dep_delay <= 0` (no delay at all) or `dep_delay <= 10`.

Everything else is clean — the use of the native pipe `|>` instead of `%>%` is perfectly fine in modern R (4.1+), and all column references match the `flights` schema correctly.

⁂

# And what is fct_reorder()?

`fct_reorder()` is a function from the `forcats` package (part of the tidyverse) that **reorders the levels of a factor based on the values of another variable**. This is especially useful in `ggplot2` charts — without it, categorical variables like airline codes or airport names appear in arbitrary alphabetical order instead of a meaningful order.[48]

### Syntax

```
fct_reorder(.f, .x, .fun = median, .desc = FALSE)
```

| Argument | Description |
|---|---|
| `.f` | The factor (or character vector) to reorder |
| `.x` | The numeric variable to sort by |
| `.fun` | Summary function applied to `.x` per level (default: `median`) |
| `.desc` | Set `TRUE` to sort in descending order |

### Why It Matters for Visualization

By default, `ggplot2` plots factor levels alphabetically. `fct_reorder()` lets you sort them by a meaningful metric, making charts far easier to read.[49]

### Example with `nycflights13`

Without `fct_reorder()`, airlines appear alphabetically in a bar chart. With it, you can sort by average delay:

```
library(forcats)

departures |>
  group_by(airline_code) |>
  summarise(mean_delay = mean(dep_delay, na.rm = TRUE)) |>
  mutate(airline_code = fct_reorder(airline_code, mean_delay)) |>
  ggplot(aes(x = airline_code, y = mean_delay)) +
  geom_col() +
  coord_flip()
```

This produces a bar chart sorted from **lowest to highest mean delay** — much more readable than alphabetical order.[48] [49]

### `.desc` Tip

To flip the order (e.g., highest delay first), add `.desc = TRUE`:

```
fct_reorder(airline_code, mean_delay, .desc = TRUE)
```

In the context of your `departures` dataframe, `fct_reorder()` would typically be used when plotting `airport` or `airline_code` against a summary metric like on-time rate, so bars or boxes are arranged in a logical order rather than alphabetically.[50]

⁂

# This is the next step in the labsheet:

Starting from departures, create a tibble departures_by_airline with one row per airline and three columns:
airline: The airline name
n: The airline's number of departures
on_time_pct: The airline's on-time departure percentage
Convert the airline names to a factor and sort its levels in ascending order of n.

How do I use on_time_pct for this case?

Your code is correct and well-structured! `on_time_pct` is calculated using `mean()` on the logical `on_time` column — since `on_time` is `TRUE`/`FALSE`, R treats `TRUE` as 1 and `FALSE` as 0, so `mean(on_time)` gives you the proportion directly.[58]

### The Full Code

```
departures_by_airline <- departures |>
  left_join(airlines, by = c("airline_code" = "carrier")) |>
  group_by(name) |>
  summarise(
    n           = n(),
    on_time_pct = mean(on_time) * 100
  ) |>
  rename(airline = name) |>
  mutate(airline = fct_reorder(airline, n))
```

### Step-by-Step Breakdown

- `left_join(airlines, ...)` — joins the `airlines` table to get the full airline name (e.g., `"United Air Lines Inc."`) from the two-letter `carrier` code[59]
- `group_by(name)` — groups by airline name so each group becomes one row
- `n = n()` — counts total departures per airline
- `on_time_pct = mean(on_time) * 100` — since `on_time` is a logical (`TRUE`/`FALSE`), `mean()` computes the fraction of `TRUE` values, and multiplying by 100 converts it to a percentage[58]

- `fct_reorder(airline, n)` — converts airline names to a factor with levels sorted in **ascending order of** `n`, as required

### Why `mean(on_time)` Works

Since you defined `on_time = dep_delay < 15` earlier, each row is either `TRUE` (1) or `FALSE` (0). So:

$$\text{on\_time\_pct} = \frac{\sum \text{TRUE values}}{\text{total rows}} \times 100$$

For example, if 800 out of 1000 flights for an airline departed within 15 minutes, `mean(on_time) * 100` gives `80` — an 80% on-time rate.

⁂

# What might be the reason one would want to use the airport name labels on the y-axis rather than the x-axis?

This is a great data visualization design question. The core reason is **readability of long text labels.**

### The Main Reason: Label Length

Airline and airport names are long strings (e.g., *"Newark"*, *"LaGuardia"*, *"United Air Lines Inc."*). When placed on the **x-axis**, these labels are horizontal by default and quickly overlap each other, forcing you to rotate them 45° or 90° — which makes them harder to read. Placing them on the **y-axis** instead means the labels sit horizontally along the left side, fully readable without any rotation.[67]

In `ggplot2`, you achieve this simply with `coord_flip()` or by mapping the categorical variable to `y` directly:

```
ggplot(departures_by_airline, aes(x = on_time_pct, y = airline)) +
  geom_col()
```

### Other Benefits

- **More space for labels** — the y-axis has more vertical room to spread out many categories without crowding[67]
- **Easier ranking comparison** — when combined with `fct_reorder()`, a horizontal bar chart lets the eye naturally scan from top to bottom to compare values, which is more intuitive for ranked lists[68]
- **Avoids diagonal text** — rotated x-axis labels (a common workaround) create extra cognitive load and are harder to read quickly[67]

This is why horizontal bar charts (categories on y-axis, values on x-axis) are the **standard recommendation** for visualizing many categorical groups — exactly the pattern your `departures_by_airline` tibble is set up for.

⁂

# For the task: Some airlines flew only a few hundred departures from New York—too few for a percentage to mean much. Remove airlines that departed fewer than 1,000 times from departures_by_airline.

Our team has already done the following:
departures_by_airline_filtered <- departures_by_airline |>
filter(n >= 1000) |>
mutate(airline = fct_reorder(airline, on_time_pct))

But when using levels(filtered_departures$airline), we still see the "SkyWest" and the other airports which has much fewer departures than 1000.

This is a classic R behaviour — **factors retain all their original levels even after rows are filtered out.** The `filter()` step removes the *rows* for SkyWest and other small airlines, but their *factor levels* remain stored in memory. [77] [78]

### The Fix

Add `fct_drop()` (from `forcats`) or `droplevels()` after filtering to remove the unused levels:

### Option 1: Using `fct_drop()` (recommended — stays in the tidyverse pipeline)

```
departures_by_airline_filtered <- departures_by_airline |>
  filter(n >= 1000) |>
  mutate(airline = fct_drop(airline)) |>
  mutate(airline = fct_reorder(airline, on_time_pct))
```

### Option 2: Using `droplevels()` (base R)

```
departures_by_airline_filtered <- departures_by_airline |>
  filter(n >= 1000) |>
  mutate(airline = droplevels(airline)) |>
  mutate(airline = fct_reorder(airline, on_time_pct))
```

After either fix, `levels(departures_by_airline_filtered$airline)` will only show the airlines with 1,000+ departures. [79] [80]

### Why This Happens

When `departures_by_airline` was first created, the `airline` factor was built with all airlines as levels. The `filter()` call drops the **rows** but not the **level metadata** — R intentionally keeps unused levels in case you need them for things like complete contingency tables. `fct_drop()` explicitly tells R to clean up levels that no longer have any associated data. [78] [79]

⁂

# Why would it be okay for a dot plot to not start at 0 on the x-axis? and on the other hand, why is it always the opposite for bar charts or lollipop charts?

This comes down to one fundamental difference: **what visual property encodes the data**.

### Bar & Lollipop Charts Encode Length

In a bar or lollipop chart, the **length of the bar/line from the baseline is the data** — your eye compares how tall or long each bar is relative to zero. If you start the axis at, say, 60% instead of 0%, a bar representing 80% looks twice as long as one representing 70%, even though the actual difference is only 10 percentage points. This is visually deceptive and has been shown in research to cause significant misinterpretation. [102] [103] [104]

The same logic applies to lollipop charts — the stick encodes length from the baseline, so truncating the axis distorts the proportional comparison between groups. [105]

### Dot Plots Encode Position

A dot plot places a dot at a **position along the axis** — there is no filled bar stretching from zero. Readers naturally interpret the dot's location, not its distance from an origin. Since no length is being visually compared to a baseline, truncating the axis doesn't distort anything — it actually **zooms in** to make meaningful differences between close values more visible. [106] [104]

### A Practical Example

In your `departures_by_airline` context, on-time percentages might all fall between 60% and 90%. On a bar chart starting at 60%, an airline at 80% would look four times "better" than one at 70% — misleading. On a dot plot starting at 60%, the dots simply show their relative positions clearly, with no implied ratio to a baseline. [107]

### The Rule of Thumb

| Chart Type | Must start at 0? | Why |
|---|---|---|
| Bar chart | ✅ Yes | Length from baseline encodes the value |
| Lollipop chart | ✅ Yes | Same as bar — stick length encodes the value |
| Dot plot | ❌ No | Position encodes the value, not length |

| Chart Type | Must start at 0? | Why |
|---|---|---|
| Line chart | ✖ No | Slope/trend is what's being read, not bar length |

A good guiding principle: **if zero isn't a meaningful or possible value in your data range, there's little justification for forcing the axis to include it** — especially for dot plots. [107]

⁂

# In this next question, could you help me to understand what it may be asking for and how do I answer it?

Find the busiest airline from the previous task—the top lollipop of the previous plot—in this punctuality ranking. Record where it appears in the ranking. The tasks that follow ask whether its raw on-time percentage should be interpreted on its own, or whether it partly reflects where and when the airline flies. Note, too, the airline at the very bottom of the ranking; you will return to it for comparison at the end.

This question is a **reading and analysis task**, not a coding task. Let me break down exactly what it's asking and how to approach it.

### What It's Actually Asking

The question has two steps across the current and future tasks:

1. **Right now:** Look at your dot plot from the previous task (the on-time percentage ranking of airlines). Find where **the busiest airline** (i.e., the one with the most departures — the top lollipop from the lollipop chart) lands in the **punctuality ranking**. Write down its position.
2. **As a preview for future tasks:** It's hinting that a raw on-time percentage can be misleading — an airline's number may look good or bad partly because of *where* and *when* it flies, not just how well it operates.

### How to Answer It

**Step 1 — Identify the busiest airline.** From your `departures_by_airline` lollipop chart, the airline with the highest `n` (the topmost lollipop, since levels were sorted by ascending `n`) is your target. In the `nycflights13` dataset, this is almost certainly **United Airlines (UA)**.

**Step 2 — Find it in the dot plot.** Your dot plot ranked airlines by `on_time_pct`. Scan the dot plot (or check the underlying tibble) to find where United Airlines sits:

```
departures_by_airline_filtered |>
  arrange(on_time_pct) |>
  mutate(rank = row_number()) |>
  select(airline, n, on_time_pct, rank)
```

**Step 3 — Note the bottom airline too.** The question explicitly asks you to remember the airline at the very bottom of the punctuality ranking (lowest `on_time_pct`). This is likely **ExpressJet** or **Mesa Airlines** — you'll need this for a comparison at the end of the lab.

### The Bigger Conceptual Point

The follow-up tasks will explore **confounding factors** — the idea that an airline flying mostly into congested airports (like Newark/EWR) or during peak morning hours will naturally have worse on-time numbers, regardless of its operational quality. This is why raw percentages shouldn't be taken at face value without context. [117]

So your answer to this task is essentially: *"The busiest airline is [X], and it ranks [Y]th out of [Z] airlines in punctuality."* Run the code above on your filtered tibble to fill in those specifics.

⁂

# In the next question:

Before taking any airline's raw punctuality rank at face value, ask a prior question: Do the airlines even fly from the same places? The New York City area has three major airports, and if they differ in their on-time records, then an airline that flies mostly from a low-punctuality airport may look worse for reasons that are not visible in the raw ranking alone. This task establishes that exposure: How each airline splits its departures across JFK, LaGuardia, and Newark.
From departures, again keep only airlines with at least 1,000 departures. Then compute, for each remaining airline, the percentage pct of its departures from each airport. As in the previous task, make airline a factor with levels in ascending order of on-time departure percentage; this ordering makes it easier to compare each airline's punctuality with the airports it flies from.

Should the data that is to be used the original data from earlier section or from the output of the previous section that already ranked them by punctuality?

You should use the **original** `departures` **tibble** — not the output from the previous section. Here's why:

### Which Data to Use

The task says *"From* `departures`*, again keep only airlines with at least 1,000 departures"* — this phrasing ("from `departures`") explicitly points you back to the raw, flight-level data. You need individual flight rows so you can count how many departures each airline made **from each specific airport**, which is impossible to compute from `departures_by_airline_filtered` since that tibble has already been summarised into one row per airline with the airport breakdown collapsed away. [127]

### What You're Building

The goal is a new tibble with one row **per airline–airport combination**, giving the percentage split of each airline's flights across JFK, LGA, and Newark:

```
departures_by_airport <- departures |>
  group_by(airline_code) |>
  filter(n() >= 1000) |>             # keep only airlines with 1000+ departures
  group_by(airline_code, airport) |>
  summarise(n_airport = n(), .groups = "drop_last") |>
  mutate(pct = n_airport / sum(n_airport) * 100) |>
  ungroup() |>
  left_join(airlines, by = c("airline_code" = "carrier")) |>
  rename(airline = name)
```

### The Factor Ordering Instruction

The task asks you to make `airline` a factor sorted by **ascending on-time percentage** — the same order as the dot plot from the previous task. You can reuse the factor levels you already computed in `departures_by_airline_filtered` rather than recalculating them:

```
departures_by_airport <- departures_by_airport |>
  mutate(airline = factor(airline,
    levels = levels(departures_by_airline_filtered$airline)
  ))
```

This ensures the airline ordering on your new chart matches the punctuality ranking exactly, making it visually easy to compare which airports each airline flies from against how punctual it is.[128]

### The Underlying Logic

The reason for going back to `departures` is the same reason you can't summarise a summary — `departures_by_airline_filtered` only has totals, not the per-airport breakdown needed here. Think of `departures` as your **single source of truth** for any new aggregation.

⁂

# Am I on the right track with this code:

```
airline_levels <- departures_by_airline_filtered |>
  arrange(on_time_pct) |>
  pull(airline) |>
  as.character()

airport_mix <- departures |>
  filter(airline %in% airline_levels) |>
  summarize(n = n(), .by = c(airline, airport)) |>
  mutate(
```

```
    pct = n / sum(n) * 100,
    .by = airline
  ) |>
  mutate(airline = factor(airline, levels = airline_levels))
```

Yes, your code is largely on the right track! There is one issue to fix and one thing worth double-checking.

### The Issue: Wrong Column Name

Your `departures` tibble (built earlier in the lab) uses `airline_code` for the carrier column, **not** `airline`. The `airline` column with full names only exists in `departures_by_airline_filtered`. So your `filter()` and `.by` grouping will silently fail or throw an error because `airline` doesn't exist in `departures`.

Fix it by referencing `airline_code` instead, then joining the full name at the end:

```
airline_levels <- departures_by_airline_filtered |>
  arrange(on_time_pct) |>
  pull(airline) |>
  as.character()

airport_mix <- departures |>
  left_join(airlines, by = c("airline_code" = "carrier")) |>  # bring in full name
  rename(airline = name) |>
  filter(airline %in% airline_levels) |>
  summarize(n = n(), .by = c(airline, airport)) |>
  mutate(
    pct = n / sum(n) * 100,
    .by = airline
  ) |>
  mutate(airline = factor(airline, levels = airline_levels))
```

### What Your Code Gets Right ✓

| Part | Assessment |
|---|---|
| `arrange(on_time_pct) \|> pull(airline) \|> as.character()` | Correct way to extract ordered factor levels as strings |
| `summarize(n = n(), .by = c(...))` | Valid modern dplyr syntax for grouped summarise |
| `mutate(pct = n / sum(n) * 100, .by = airline)` | Correctly computes within-airline percentages so each airline's airports sum to 100% |
| `factor(airline, levels = airline_levels)` | Correctly applies the punctuality-based ordering |

### Quick Sanity Check

After running it, verify each airline's percentages sum to 100:

```
airport_mix |>
  summarise(total_pct = sum(pct), .by = airline)
```

All values should be exactly `100`.

⁂

# Could you teach me again how summarize works? and pct

Great question! Let me walk you through both concepts clearly using your actual code as the example.

### How `summarize()` Works

`summarize()` (or `summarise()`) **collapses many rows down into fewer rows** by applying an aggregation function. It's always used together with a grouping — either via `group_by()` beforehand, or the modern inline `.by =` argument.

Think of it this way:

```
Before summarize:                After summarize(n = n(), .by = c(airline, airport))
airline    airport  on_time      airline         airport   n
"United"   "EWR"    TRUE    →    "United"        "EWR"     4000
"United"   "EWR"    FALSE        "United"        "JFK"     2000
"United"   "JFK"    TRUE         "Delta"         "JFK"     5000
"Delta"    "JFK"    TRUE         ...
...
```

In your code:

```
summarize(n = n(), .by = c(airline, airport))
```

- `.by = c(airline, airport)` — groups every unique **airline + airport combination** together
- `n = n()` — counts how many rows (flights) are in each group
- The result has **one row per airline–airport pair**

### How `pct` Works

After `summarize()`, you have counts `n` per airline–airport pair. The next `mutate()` converts those counts into **percentages within each airline**:

```
mutate(
  pct = n / sum(n) * 100,
  .by = airline
)
```

The `.by = airline` means `sum(n)` is calculated **separately for each airline** — so it sums only that airline's airport counts, not the entire dataset. Here's the arithmetic:

```
airline    airport   n      sum(n) for airline   pct
"United"   "EWR"     4000   ÷ 8000  × 100  =     50%
"United"   "JFK"     2000   ÷ 8000  × 100  =     25%
"United"   "LGA"     2000   ÷ 8000  × 100  =     25%
"Delta"    "JFK"     5000   ÷ 7000  × 100  =     71.4%
...
```

Each airline's three airport percentages will always **add up to 100%**, which is exactly what you want — it shows the *share* of each airline's departures that came from each airport.

### The Key Distinction

| Function | What it does | Rows in → Rows out |
|---|---|---|
| `summarize()` | Collapses rows, computes aggregates | Many → Few |
| `mutate()` | Adds/transforms columns, keeps all rows | Same count |

In your pipeline, `summarize()` first reduces the data to one row per airline–airport pair, then `mutate()` adds the `pct` column to those already-summarised rows.

⁂

# I do not understand the next question that is being asked in the lab sheet:

Is the dominant departure airport of the busiest airline also the airport with the lowest on-time percentage? If so, what does that suggest? Could the airline’s raw rank partly reflect where it flies rather than its schedule-adjusted punctuality?

Could you explain to me what it is asking for and how should i approach answering it?

This is a **conceptual reasoning question** — it's not asking you to write new code. It wants you to connect two things you've already computed and draw a conclusion. Here's a full breakdown.

### What It's Actually Asking

The question has three linked parts:

**Part 1: "Is the dominant departure airport of the busiest airline also the airport with the lowest on-time percentage?"**

Look at your `airport_mix` tibble. For the busiest airline (United Airlines), find which airport has the highest `pct` — that's its "dominant" airport. Then separately check which of the three NYC airports (EWR, JFK, LGA) has the lowest overall on-time rate. You can compute this quickly:

```
# On-time rate by airport across all airlines
departures |>
  group_by(airport) |>
  summarise(on_time_pct = mean(on_time) * 100)
```

In the `nycflights13` data, **Newark (EWR) is historically the worst-performing airport** for on-time departures. United Airlines predominantly flies out of EWR. So the answer to Part 1 is likely **yes.**

**Part 2: "If so, what does that suggest?"**

This is where you write a short interpretive sentence. The logic is:

- United flies mostly from EWR
- EWR has a poor on-time record (due to congestion, airspace constraints, etc.)
- Therefore, United's low punctuality ranking may be **partly caused by the airport it operates from**, not purely by how well United manages its own schedule

**Part 3: "Could the airline's raw rank partly reflect where it flies rather than its schedule-adjusted punctuality?"**

This is asking you to agree or disagree with a hypothesis. The answer is **yes** — this is the concept of a **confounding variable**. The airport is a confound: it affects on-time performance independently of the airline's own operational quality.

### How to Structure Your Answer

Think of it as three sentences:

1. **Observation** — "United Airlines departs predominantly from Newark (EWR), which accounts for approximately X% of its flights."
2. **Connection** — "Newark has the lowest on-time departure percentage among the three NYC airports at Y%."
3. **Conclusion** — "This suggests United's raw punctuality rank may partly reflect the poor on-time environment at Newark rather than its intrinsic scheduling performance — meaning a fairer comparison would need to account for which airports each airline flies from."

### The Bigger Picture

This question is building toward the concept of **Simpson's Paradox** and **confounding** in statistics. An airline could actually be *better* at managing its own departures than a competitor, yet still appear worse in a raw ranking simply because it's stuck flying from a chronically delayed airport. The lab is training you to ask *"what else might explain this number?"* before drawing conclusions from a single metric.

⁂

# Would this help to answer the question?

Yes, these two charts together are **exactly what you need** to answer the question fully. Here's how to use them:

### Reading the Two Charts Together

**From the dot plot**, United Airlines sits **6th out of 11** in punctuality at **78.2%** on-time — right in the middle of the ranking, while the busiest airline in terms of departures is United. [166]

**From the stacked bar chart**, United's dominant airport is clearly **Newark (blue) at 78.7%** of its departures — meaning nearly 4 in 5 United flights left from EWR. [167]

### Answering the Three Parts

**Part 1 — Is the dominant airport also the worst for on-time performance?**
Yes. United flies predominantly from Newark (78.7% of departures), and Newark is known to be the worst-performing of the three NYC airports — you can verify this by running `departures |> group_by(airport) |> summarise(mean(on_time) * 100)`.

**Part 2 — What does that suggest?**
United's middling rank of 6th is likely dragged down by its heavy reliance on Newark. An airline that flies 78.7% of its flights from the worst-performing NYC airport is structurally disadvantaged in any raw punctuality comparison.

**Part 3 — Does the raw rank reflect where it flies more than its own punctuality?**
Yes — this is a strong case for that argument. Compare United (78.2%, mostly Newark) against **US Airways** at the top (87.4%), which flies **63.3% from LaGuardia** — historically the best-performing NYC airport. The difference in ranking may say more about airport choice than airline quality.

### The Bottom Airline

Also note from the dot plot: **ExpressJet is last at 68.9%**, and from the bar chart it flies **81.3% from Newark**. This is the airline you were asked to note for comparison at the end of the lab — it's an even more extreme version of the same pattern, flying almost entirely from the most delay-prone airport. [167] [166]

⁂

# Could you help me on how I should approach this question:

From departures, compute for each airline and each scheduled hour the percentage pct of that airline's own departures falling in that hour, so that each airline's hourly percentages sum to 100%. An hour in which an airline operates no departures has a genuine share of 0%, not a missing value, so make sure every airline has a row for every hour in the data (i.e., from 5 to 23): Add the absent hours back with a count of zero—for example with tidyr::complete()—before computing the percentages. Filling in these zeros leaves every airline's total unchanged, so the percentages are unaffected.

Great question — this is a multi-step pipeline that introduces a new function. Let me walk through the whole thing clearly.

### What the Question Wants

The final output should be a tibble with **one row per airline–hour combination** (e.g., United at hour 6, United at hour 7, ...), where `pct` tells you what share of that airline's total departures happened in that hour. Every airline must have all hours from 5 to 23 represented, even if they flew zero flights in some of those hours.

### The Pipeline — Step by Step

### Step 1: Filter to airlines with 1,000+ departures

Same as before — use a `group_by` + `filter(n() >= 1000)` approach on `departures`:

```
library(tidyr)

hour_mix <- departures |>
  group_by(airline_code) |>
  filter(n() >= 1000) |>
  ungroup()
```

### Step 2: Count departures per airline–hour

```
hour_mix <- hour_mix |>
  summarise(n = n(), .by = c(airline_code, hour))
```

At this point you only have rows for hours where each airline **actually flew**. An airline with no 9am flights simply has no row for hour 9. This is the **implicit missing value** problem.

### Step 3: Fill in the missing hours with `complete()`

`tidyr::complete()` generates all **combinations** of variables you specify, and the `fill` argument sets what value to use for missing rows:[168]

```
hour_mix <- hour_mix |>
  complete(
    airline_code,
    hour = 5:23,        # explicitly define the full range of hours
    fill = list(n = 0)  # absent hours get a count of 0, not NA
  )
```

After this, every airline has exactly 19 rows (hours 5 through 23), with `n = 0` for any hour they didn't fly.[169] [168]

### Step 4: Compute the percentage within each airline

```
hour_mix <- hour_mix |>
  mutate(
    pct = n / sum(n) * 100,
    .by = airline_code
  )
```

Because the zeros don't change each airline's `sum(n)` (adding zero changes nothing), the percentages are identical to what you'd get without the zero-filling — but now every airline has a complete row for every hour.[169]

### Step 5: Join in airline names and apply factor ordering

```
hour_mix <- hour_mix |>
  left_join(airlines, by = c("airline_code" = "carrier")) |>
  rename(airline = name) |>
  mutate(airline = factor(airline, levels = airline_levels))
```

### Why `complete()` Matters Here

Without it, a `ggplot2` heatmap or faceted chart would either skip missing hours or treat them as `NA` (which plots as grey/blank) rather than a genuine 0%. By making the zeros **explicit**, your visualization correctly shows an airline simply didn't fly at that hour — which is meaningful information, not missing data.[170]

| Without `complete()` | With `complete()` |
|---|---|
| Missing rows for hours with 0 flights | A row exists for every airline–hour |
| `NA` appears in calculations/plots | `0` correctly represents no flights |
| Percentages still sum to 100% | Percentages still sum to 100% |

⁂

# Could you further explain how tidyr::complete() works?

Sure! Let me walk through `tidyr::complete()` from the ground up.

### The Core Problem It Solves

When you `summarise()` your data, you only get rows for combinations that **actually exist**. If United Airlines flew zero flights at hour 22, there's simply no row for `(United, 22)` in your result. This is called an **implicit missing value** — it's missing, but R doesn't tell you it's missing. [193]

`complete()` turns those implicit missing values into **explicit** ones by generating all the combinations you expect to be there. [193]

### A Simple Mental Model

Imagine you have this after `summarise()`:

```
airline    hour    n
"United"   6       120
"United"   7       340
"United"   9       200
            ↑ hour 8 is missing entirely!
```

After `complete(airline, hour = 5:23, fill = list(n = 0))`:

```
airline    hour    n
"United"   5       0   ← added
"United"   6       120
"United"   7       340
"United"   8       0   ← added
"United"   9       200
"United"   10      0   ← added
...and so on up to hour 23
```

### Anatomy of the Function

```
complete(
  data,
  airline_code,        # first variable to expand
  hour = 5:23,         # second variable — explicitly define the full range
  fill = list(n = 0)   # what to put in the 'n' column for new rows
)
```

| Argument | What it does |
|---|---|
| `airline_code` | Uses all unique airlines already in the data |
| `hour = 5:23` | Forces every hour from 5 to 23 to appear |
| `fill = list(n = 0)` | New rows get `n = 0` instead of NA |

The result is every **airline × hour** combination, including ones with no flights.[193]

### Why `fill = list(n = 0)` and Not Just `NA`?

By default, `complete()` fills new rows with `NA`. But in your case, a missing hour means **zero flights**, not unknown data. If you left it as `NA`, then `sum(n)` and `mean(on_time)` would return `NA` for any airline with a gap, breaking your percentage calculation.[194]

Setting `fill = list(n = 0)` means:

- `sum(n)` is unaffected (adding 0 changes nothing ✓)
- `pct = n / sum(n) * 100` correctly gives 0% for empty hours ✓
- Your plot shows a genuine empty bar/cell, not a grey "missing" tile ✓

### `nesting()` — a Bonus Concept

Sometimes you don't want *all* combinations of two variables — you want to expand one variable while keeping existing pairs of two others intact. That's what `nesting()` does inside `complete()`:[193]

```
# Expand all groups × all hours, but keep item_id and item_name as a fixed pair
complete(group, nesting(item_id, item_name), fill = list(value = 0))
```

In your lab, you don't need `nesting()` because `airline_code` and `hour` are independent — you genuinely want every airline paired with every hour.[193]

### Full Picture in Your Pipeline

```
hour_mix <- departures |>
  group_by(airline_code) |>
  filter(n() >= 1000) |>
  ungroup() |>
  summarise(n = n(), .by = c(airline_code, hour)) |>
  # ↓ This is where complete() fills in the gaps
  complete(airline_code, hour = 5:23, fill = list(n = 0)) |>
  mutate(pct = n / sum(n) * 100, .by = airline_code)
```

The `complete()` step sits **between** the count and the percentage calculation — after you know the counts, but before you divide, so the denominator (`sum(n)`) is always correct.[195]

⁂

# From the code snippet in my Section 3.3b:

filtered_departures <- filtered_departures |> mutate(airline = fct_drop(airline)) |> mutate(airline = fct_reorder(airline, on_time_pct))

How can I draw a heatmap for it? Do let me know how I can control what is on the x and y axis

A heatmap in `ggplot2` is built using `geom_tile()`, where the **x and y axes are your two categorical variables** and the **fill colour represents the numeric value** (in this case `pct`). [204]

### The Three Things a Heatmap Needs

Every heatmap tile needs exactly three mapped aesthetics: [204]

| Aesthetic | Maps to | In your case |
|---|---|---|
| x | Horizontal axis | `hour` (scheduled departure hour) |
| y | Vertical axis | `airline` (factor, ordered by on-time %) |
| `fill` | Tile colour | `pct` (% of airline's departures in that hour) |

### The Code

Assuming your `hour_mix` tibble is built (with `complete()` and the factor ordering applied):

```
ggplot(hour_mix, aes(x = hour, y = airline, fill = pct)) +
  geom_tile() +
  scale_fill_gradient(low = "white", high = "steelblue") +
  labs(
    title = "Share of Departures by Hour for Each Airline (2013)",
    x = "Scheduled Departure Hour",
    y = NULL,
    fill = "% of Departures"
  ) +
  theme_minimal()
```

### Controlling the Axes

**Swapping x and y** is as simple as swapping which variable you put in `aes(x = ...)` and `aes(y = ...)`: [205]

```
# Hour on y-axis, airline on x-axis (less readable for long names)
aes(x = airline, y = hour, fill = pct)

# Hour on x-axis, airline on y-axis (recommended — labels are readable)
aes(x = hour, y = airline, fill = pct)
```

The second layout is better here for exactly the same reason discussed earlier — airline names are long and read naturally when placed horizontally on the y-axis. [204]

### Customising the Colour Scale

You have three good options for `fill`:[204]

```
# Option 1: Simple white-to-colour gradient
scale_fill_gradient(low = "white", high = "steelblue")

# Option 2: ColorBrewer palette
scale_fill_distiller(palette = "Blues", direction = 1)

# Option 3: Viridis (colourblind-friendly)
scale_fill_viridis_c()
```

### Why `airline` is on the y-axis here

Since `airline` was converted to a factor with levels ordered by `on_time_pct` (from `fct_reorder()`), placing it on the y-axis means airlines are arranged from **worst to best punctuality going upward** — making it visually easy to spot whether low-ranked airlines cluster their departures in certain peak hours that might explain their poor on-time performance.[206]

⁂

# This is how I ended up doing it:

airline_performance <- departures |>
summarize(on_time_pct = mean(on_time) * 100, .by = airline)

airline_hourly_pct <- airline_hourly_pct |>
left_join(airline_performance, by = "airline") |>
mutate(airline = fct_reorder(airline, on_time_pct))

ggplot(airline_hourly_pct, aes(x = hour, y = airline, fill = pct)) +
geom_tile( linewidth = 0.2) +
scale_x_continuous(breaks = seq(5, 23, by = 2)) +
departure_labs(
title = "Hourly Flight Distribution by Airline",
x = "Scheduled Departure Hour",
y = "Airline",
fill = "Share of Daily Flights"
)

Is this okay?

Your code is mostly correct and well-structured! There is one small issue to flag.

### The Issue: `departure_labs()` → `labs()`

`departure_labs()` is not a real `ggplot2` function — this should be `labs()`. Everything else about your label setup is correct, just change that one word:[219]

```
labs(
  title = "Hourly Flight Distribution by Airline",
  x = "Scheduled Departure Hour",
  y = "Airline",
  fill = "Share of Daily Flights"
)
```

### What Your Code Gets Right ✅

| Part | Assessment |
|---|---|
| `summarize(on_time_pct = mean(on_time) * 100, .by = airline)` | Correctly computes per-airline on-time % for ordering |
| `left_join(airline_performance, by = "airline")` | Correctly brings in `on_time_pct` so `fct_reorder` has something to sort by |
| `fct_reorder(airline, on_time_pct)` | Correctly orders airlines by punctuality on the y-axis |
| `aes(x = hour, y = airline, fill = pct)` | Correct axis mapping for a readable heatmap |
| `scale_x_continuous(breaks = seq(5, 23, by = 2))` | Clean touch — shows every other hour instead of cluttering the axis |
| `geom_tile(linewidth = 0.2)` | Adds a subtle grid between tiles for readability |

### One Optional Improvement

You may want to add a colour scale explicitly, otherwise `ggplot2` uses its default blue gradient. A common choice for percentage share data is:

```
scale_fill_distiller(palette = "Blues", direction = 1)
```

This ensures low percentages map to light colours and high percentages to dark, making peak departure hours stand out clearly.[219]

⁂

# The busiest airline seen in previous sections (that you have not seen), was United airlines as it had the highest number of departures of 57979.

Does it concentrate its departures in late, lower-punctuality hours, or does it fly at morefavorable times? Based on the heatmap, how much can when it flies explain its raw punctuality rank

United airlines had a on-time departure rate of 78.2 in section 3.3, being ranked 6th

Looking at the heatmap, here is a structured analysis you can use to answer this question. [220]

### What the Heatmap Shows for United Airlines

United Airlines' row shows its departures are **broadly spread across hours 6 to 21**, with no single dominant peak hour. There is a moderate concentration in the **morning block (hours 6–9)** and again in the **afternoon (hours 14–18)**, but no overwhelming spike in any one slot.

### Does United Fly at Late, Lower-Punctuality Hours?

The answer is **no, not disproportionately.** Research consistently shows that early morning flights (hours 6–9) tend to have the best on-time performance because delays haven't yet cascaded through the day's operations. United does operate flights in these favorable early hours. It also has departures spread through mid-afternoon, which is a moderately higher-delay period, but nothing extreme compared to airlines that are heavily concentrated in evening hours. [221]

### How Much Does Timing Explain United's Rank?

Timing explains **relatively little** of United's 6th-place ranking of 78.2%. Compare this to the airport exposure finding from earlier: United flies **78.7% of its departures from Newark (EWR)** — the worst-performing NYC airport. That structural disadvantage is far more impactful than its hourly schedule. [222]

In contrast, look at airlines like **Mesa** and **Frontier** in the heatmap — they show very sharp, concentrated spikes at specific hours, meaning their punctuality numbers are more likely to be influenced by *when* they fly. United's spread is actually fairly typical and balanced. [220]

### Your Answer in Three Points

1. **United does not concentrate its flights in late, low-punctuality hours** — its schedule is spread across the day with a modest morning bias, which is actually a *favorable* timing pattern [221]
2. **Timing is therefore not a strong explanation** for its middling 6th-place rank — if anything, its hourly distribution should be helping it slightly, not hurting it
3. **The more compelling explanation remains airport exposure** — Newark's structural delay environment (established in the previous section) is a much stronger confound than

departure timing

This means United's 78.2% on-time rate is likely a more genuine reflection of its operating environment (Newark) than an artefact of flying at bad hours.

⁂

# Is the 6th hour to the 20th hour favourable times? If so, or isn't explain why as of whether the heatmap can explain its punctuality rank

Great question — and the data gives a clear, nuanced answer. Let me break down the full hour range.

### The Punctuality Spectrum by Hour

Research on US domestic flights shows on-time performance **declines steadily as the day progresses** — this is called the **delay cascade effect**, where small hiccups from early flights snowball into larger delays later: [237] [238]

| Hour Range | On-Time Rate | Verdict |
|---|---|---|
| 6am – 10am | ~88–90% | ✅ Most favourable — fresh aircraft, low congestion |
| 10am – 3pm | ~80–86% | ⚠️ Moderate — still decent, delays start building |
| 3pm – 6pm | ~76–80% | ⚠️ Less favourable — delay accumulation accelerating |
| 6pm – 11pm | ~73–75% | ❌ Least favourable — cascaded delays at their worst |

^20_2

### Applying This to United's Heatmap

Looking at United's row in the heatmap, its departures are **spread fairly evenly from hours 6 to roughly 20**, with no strong concentration in either the best or worst zone. This means: [239]

- United **does** operate in the favorable early hours (6–10) — which should help its score
- But it also has a **meaningful spread into hours 14–20**, which are moderate-to-unfavourable — a slight drag on its raw rate
- Crucially, it has **very little concentration** in the worst hours (20–23), unlike some other airlines

### Can Timing Explain United's 6th-Place Rank?

**Only partially, and weakly.** Here's the reasoning:

1. United's hourly spread is **fairly typical** — not dramatically skewed toward bad hours compared to other airlines in the heatmap [239]

2. Airlines like **Hawaiian and US Airways** (top of the ranking) appear to have **tighter concentrations in morning hours**, which likely helps their numbers
3. But United's **dominant confound remains Newark (EWR)** — flying 78.7% of flights from the worst NYC airport is a far stronger structural disadvantage than any hour-of-day effect [240]

In short: the timing of United's flights is neither unusually good nor unusually bad — it **does not meaningfully explain** why United sits 6th. The airport exposure story from the previous section remains the more compelling explanation for its raw rank.

⁂

# The below is my answer for the question:

Interpret the heatmap. Locate the busiest airline you have been tracking since Section 3.3. Does it concentrate its departures in late, lower-punctuality hours, or does it fly at more favorable times? Based on the heatmap, how much can when it flies explain its raw punctuality rank?

My answer:
Looking at the heatmap, United airlines seem to have consistent flights from the 6th hour all the way to the 20th hour. The early morning hours tend to be favourable times while the afternoon/evening hours tend to be more unfavourable times. However, this heatmap does not explain the punctuality rank of United airlines as US Airways also has a similar flight schedule but achieves a 87.4% on-time departure rate to United's 78.2% on-time departure rate

Can I have your opinions on it?

Your answer is a good start and shows genuine reasoning, but it has a few gaps that could make it stronger. Here's my honest assessment:

### What You Got Right ✅

- Correctly observing that United flies **consistently from hours 6 to 20** — this is an accurate reading of the heatmap [253]
- Correctly noting that **early hours are favourable and later hours less so** — this shows good understanding of the delay cascade principle [254]
- The US Airways comparison is a **smart intuition** — noticing that a similar spread of hours produces a very different on-time rate is exactly the kind of critical thinking the question is looking for

### What Could Be Stronger 🔲

**1. The conclusion is slightly incomplete.**
You said the heatmap "does not explain" the punctuality rank, but you haven't fully explained *why*. The key reasoning step is:

> If the heatmap were the explanation, then airlines with similar hourly spreads should have similar on-time rates. But US Airways and United have comparable schedules yet very different rates — which means *something else* (airport exposure) must be the real driver.

This is your argument, but you should state it explicitly rather than leaving it implied.

**2. You haven't connected back to the airport finding.**
The question is designed to build on the previous section. Your answer would be much stronger by explicitly referencing it:

> *"Since United's hourly distribution is not unusually skewed toward bad hours, the timing of its flights alone cannot explain its 6th-place rank. A stronger explanation, established in the previous section, is that United operates 78.7% of its flights from Newark — the worst-performing NYC airport."*

### Suggested Revised Answer

> United Airlines flies consistently from hours 6 through 20, with no strong concentration in the late-night hours where delays are most severe. While early morning departures (hours 6–10) tend to have the best on-time performance due to fewer cascading delays, United's spread is not dramatically different from higher-ranked airlines such as US Airways, which achieves an 87.4% on-time rate despite a similar hourly distribution. This suggests that *when* United flies explains little of its 6th-place rank of 78.2%. A more compelling explanation, established in the previous section, is United's heavy reliance on Newark (EWR) — the worst-performing NYC airport — for 78.7% of its departures.

This ties together all three tasks (punctuality rank, airport mix, hourly mix) into a coherent argument, which is likely what the lab is building toward.

⁂

# This is a code snippet from the labsheet:

First, compute the slot baseline: Across all of departures, calculate the on-time percentage for each airport-and-hour slot.

```
slots <-
departures |>
summarize(
n = n(),
baseline_on_time_pct = mean(on_time) * 100,
.by = c(airport, hour)
)
slots
```

Then, for each airline with at least 1,000 departures, assemble its number of departures n, its observed on_time_pct, its expected_on_time_pct (the average of the airport-hour baselines over that airline's own departures), and its dominant_airport—the airport from which it departs

most often, as identified in Section 3.4.

```
airline_performance <-
departures |>
left_join(slots, by = c("airport", "hour")) |>
summarize(
n = n(),
on_time_pct = mean(on_time) * 100,
expected_on_time_pct = mean(baseline_on_time_pct),
.by = airline
) |>
filter(n >= 1000) |>
left_join(
departures |>
count(airline, airport) |>
slice_max(n, n = 1, by = airline, with_ties = FALSE) |>
select(airline, dominant_airport = airport),
by = "airline"
)
airline_performance
```

Could you explain what is being done in the code above?

This code does something conceptually rich — it computes a **schedule-adjusted expected punctuality** for each airline. Let me walk through each block.

### Block 1: Computing the Slot Baseline (`slots`)

```
slots <- departures |>
  summarize(
    n = n(),
    baseline_on_time_pct = mean(on_time) * 100,
    .by = c(airport, hour)
  )
```

This creates a **reference table** — one row per unique **airport + hour combination** (e.g., EWR at 7am, JFK at 14pm, etc.), with the average on-time rate across **all airlines** flying in that slot.

Think of it as asking: *"Regardless of which airline it was, how often did flights from EWR at 7am depart on time?"*

This becomes the **baseline** — the expected punctuality for any flight in that slot, purely based on where and when it departs, with no airline identity considered.

### Block 2: Computing Airline Performance (`airline_performance`)

This block has three distinct steps chained together:

### Step 1 — Join the slot baseline onto every individual flight

```
departures |>
  left_join(slots, by = c("airport", "hour"))
```

Each flight row now carries its slot's `baseline_on_time_pct`. A United flight from EWR at 7am gets tagged with the on-time rate for *all* flights from EWR at 7am.

### Step 2 — Summarise per airline

```
summarize(
  n = n(),
  on_time_pct = mean(on_time) * 100,
  expected_on_time_pct = mean(baseline_on_time_pct),
  .by = airline
)
```

Three things are computed per airline:

| Column | What it means |
|---|---|
| `n` | Total number of departures |
| `on_time_pct` | The airline's **actual** observed on-time rate |
| `expected_on_time_pct` | The **average slot baseline** across all that airline's flights — what any airline flying the same schedule would be expected to achieve |

The key insight: `expected_on_time_pct` is a **counterfactual.** It answers: *"If an average airline flew United's exact schedule (same airports, same hours), what on-time rate would it get?"*

### Step 3 — Join in the dominant airport

```
left_join(
  departures |>
    count(airline, airport) |>
    slice_max(n, n = 1, by = airline, with_ties = FALSE) |>
    select(airline, dominant_airport = airport),
  by = "airline"
)
```

This sub-pipeline:

- Counts how many flights each airline flew from each airport
- Uses `slice_max()` to keep only the airport with the **highest count** per airline

- Joins that dominant airport label back onto `airline_performance`

`with_ties = FALSE` ensures that if two airports are tied, only one row is kept (no duplicates).

### The Big Picture

After this code, `airline_performance` has one row per airline with:

```
airline | n | on_time_pct | expected_on_time_pct | dominant_airport
```

The comparison between `on_time_pct` and `expected_on_time_pct` is what makes this powerful:

- If `on_time_pct ≈ expected_on_time_pct` → the airline performs **as expected** for its schedule; its rank is mostly explained by *where and when* it flies
- If `on_time_pct > expected_on_time_pct` → the airline **outperforms** its schedule; it's genuinely better than its conditions would predict
- If `on_time_pct < expected_on_time_pct` → the airline **underperforms** its schedule; it struggles even relative to its own slots

For United specifically, this will let you see whether its 78.2% is what you'd expect given its heavy use of Newark — or whether it's actually doing better or worse than that exposure predicts.

⁂

# I would need to make a bubble plot for the previous mentioned code snippet. I need to configure the x-axis, y-axis, point area using scale_size_area(), the point color mapping and each airline labelled with geom_text_repel().

Could you help to give the code structure which I can use to do this? Do provide the explanation for it

Here is the full code structure with explanations for each component.

### The Code

```
library(ggrepel)

ggplot(airline_performance,
       aes(x = expected_on_time_pct,
           y = on_time_pct,
           size = n,
           colour = dominant_airport,
           label = airline)) +
```

```
  geom_abline(slope = 1, intercept = 0,
              linetype = "dashed", colour = "grey50") +
  geom_point(alpha = 0.7) +
  geom_text_repel(size = 3,
                  max.overlaps = Inf,
                  box.padding = 0.5) +
  scale_size_area(max_size = 15,
                  name = "Number of Departures") +
  scale_colour_brewer(palette = "Set1",
                      name = "Dominant Airport") +
  labs(
    title = "Observed vs. Expected On-Time Departure Rate by Airline",
    x = "Expected On-Time % (based on airport-hour slots)",
    y = "Observed On-Time %",
  ) +
  theme_minimal()
```

### Explanation of Each Part

#### The `aes()` Mappings

| Aesthetic | Variable | Why |
|---|---|---|
| `x` | `expected_on_time_pct` | What punctuality the airline's schedule predicts |
| `y` | `on_time_pct` | What the airline actually achieved |
| `size` | `n` | Bubble area proportional to number of departures |
| `colour` | `dominant_airport` | Reveals which airport each airline mainly uses |
| `label` | `airline` | Text to be placed by `geom_text_repel()` |

#### The Reference Line: `geom_abline()`

```
geom_abline(slope = 1, intercept = 0, linetype = "dashed", colour = "grey50")
```

This draws a **diagonal y = x line** — the most important element of the plot. Points **above** the line mean the airline outperforms its expected rate; points **below** mean it underperforms. Without this line the chart loses its key interpretation layer. [274]

#### `scale_size_area()` vs `scale_size()`

```
scale_size_area(max_size = 15, name = "Number of Departures")
```

`scale_size_area()` ensures that the **visual area** of each bubble is proportional to `n`, and crucially that `n = 0` maps to a point of zero area. This is more honest than `scale_size()`, which maps to radius instead of area — an airline with twice the departures gets a bubble twice as

large in area, not just twice the radius (which would look four times bigger). Adjust `max_size` to taste.[275]

`geom_text_repel()`

```
geom_text_repel(size = 3, max.overlaps = Inf, box.padding = 0.5)
```

This automatically positions airline name labels so they don't overlap each other or the bubbles:[276]

- `max.overlaps = Inf` — forces all labels to be shown even in crowded areas[277]
- `box.padding = 0.5` — adds space between the label text and its anchor point
- It draws thin connecting lines from labels to their bubbles when the label has been moved far away[278]

`scale_colour_brewer()`

```
scale_colour_brewer(palette = "Set1", name = "Dominant Airport")
```

Maps `dominant_airport` (EWR, JFK, LGA) to three distinct colours from the ColorBrewer "Set1" palette, which is designed for categorical data with good contrast.[279]

### Reading the Final Plot

Once rendered, you can answer the core question directly:

- **United Airlines** — if its bubble sits **below the diagonal**, its observed rate is worse than its schedule predicts, meaning factors beyond airport/hour are hurting it
- **ExpressJet** — the bottom-ranked airline; check whether it sits on, above, or below the diagonal to see if Newark explains its poor ranking or if it genuinely underperforms its own slots

⁂

# How do I:

Add the line of equality with geom_abline(slope = 1, intercept = 0)

`geom_abline()` is a simple addition to your `ggplot()` chain. Here is everything you need to know.

### What It Does

`geom_abline(slope = 1, intercept = 0)` draws the **line of equality** — the diagonal line where y = x. On your bubble plot, this means any airline sitting **exactly on the line** has an observed on-time rate that perfectly matches its expected rate. It is the most important visual reference on the chart.[289]

### How to Add It

Simply drop it into your `ggplot()` chain **before** `geom_point()` so the bubbles are drawn on top of the line:

```
ggplot(airline_performance,
       aes(x = expected_on_time_pct,
           y = on_time_pct,
           size = n,
           colour = dominant_airport,
           label = airline)) +
  geom_abline(slope = 1, intercept = 0,          # ← line of equality
              linetype = "dashed",
              colour = "grey50") +
  geom_point(alpha = 0.7) +
  geom_text_repel(size = 3, max.overlaps = Inf)
```

### The Three Arguments

| Argument | What it controls |
|---|---|
| `slope = 1` | The line rises 1 unit in y for every 1 unit in x — a perfect 45° diagonal |
| `intercept = 0` | The line passes through the origin (0, 0) |
| `linetype = "dashed"` | Makes it a dashed line so it doesn't visually compete with the data points |

### Important: Order in the Chain Matters

`geom_abline()` should come **before** `geom_point()` in your code. ggplot2 draws layers in order from top to bottom, so placing the line first ensures the bubbles appear on top of it — making the chart cleaner and easier to read.[289]

### Reading the Result

Once added, interpreting the chart becomes straightforward:

- **Above the line** → airline outperforms its schedule (better than expected)
- **Below the line** → airline underperforms its schedule (worse than expected)
- **On the line** → airline performs exactly as its airport-hour slots predict

For United Airlines specifically, check whether its bubble falls above or below — this directly answers whether its 6th-place rank is explained by its Newark exposure, or whether there is something in its own operations dragging it down further.

⁂

# For the question:

a) Find the busiest airline you have tracked since Section 3.3 (United) on the bubble plot.

b) Compare its observed and expected on-time percentages. Does its position relative to the line of equality change how you interpret its raw punctuality rank? In your answer, refer to its dominant airport and to the airport-hour baseline represented on the x-axis.

c) Now return to the airline at the very bottom of the punctuality ranking, the one you noted earlier (ExpressJet). Compare its dominant airport, expected on-time percentage, and position relative to the equality line with those of the busiest airline. Are the two airlines facing similar airport-hour circumstances? If so, what does their difference in observed on-time percentage suggest?

Reading directly from the bubble plot, here are model answers for all three parts.[299]

### Part a) — Locating United Airlines

United Airlines is the **large blue bubble** on the left side of the plot, positioned at approximately:[299]

- **Expected on-time** % ≈ 75.5% (x-axis)
- **Observed on-time** % ≈ 78.5% (y-axis)

Its bubble is the largest blue one, reflecting its status as the busiest airline with 57,979 departures. Its dominant airport is **Newark (EWR)**, indicated by the blue colour.

### Part b) — United's Position Relative to the Line of Equality

United sits **above the dashed line of equality**, meaning its observed on-time rate (~78.2%) is **higher than its expected rate** (~75.5%) based on the airport-hour slot baselines.[299]

This meaningfully changes how we interpret its raw 6th-place rank. The x-axis baseline tells us that flights from United's slots — predominantly Newark at various hours — are structurally expected to achieve only around 75.5% on-time performance. United actually **outperforms** that expectation by roughly 2–3 percentage points. So rather than being a middling performer, United is in fact **operating better than its Newark-heavy schedule would predict.** Its raw rank undersells its actual operational quality because it is penalised by its dominant airport's poor baseline.

### Part c) — Comparing United and ExpressJet

Looking at the plot, ExpressJet is the **small blue bubble** in the bottom-left corner, with:[299]

- **Expected on-time** % ≈ 75.5% (x-axis) — almost identical to United
- **Observed on-time** % ≈ 69% (y-axis) — far below United
- **Dominant airport**: also **Newark (EWR)** (blue colour)

The two airlines face **nearly identical airport-hour circumstances** — both are Newark-dominant, and both have an expected on-time rate of around 75.5%. This makes them a near-perfect natural comparison pair.

Yet their observed rates differ by roughly **9 percentage points** (78.2% vs 69%), and ExpressJet sits **well below the line of equality** while United sits above it. This stark difference in position relative to the diagonal, despite facing the same structural conditions, suggests that **ExpressJet genuinely underperforms its own slots** — its poor raw ranking is not just an artefact of flying from Newark, but reflects real operational weaknesses beyond what the airport-hour environment would explain. United, by contrast, manages to beat those same conditions. [299]

⁂

# This is the answer that my team wrote up instead:

United has an expected on-time percentage of roughly ~75.8%, and an observed on-time percentage of about 78.2%. This means that United airlines is actually more on-time than expected.
Looking at the line of equality, United should at least be ranked 5th instead of 6th in the raw punctuality rank. The 5th ranked airline in terms of punctuality was Envoy, but Envoy falls below the line of equality. Additionally, United's dominant airport is Newark. Looking at the x-axis, the airlines that are dominantly Newark generally cluster on the far left, meaning Newark has the lowest "airport-hour baseline" expectations in the dataset.
Looking at ExpressJet which was at the bottom of the punctuality ranking, ExpressJet has an expected on-time percentage of roughly ~75.2%, but its observed percentage is far lower, at 68.9%. Like United, ExpressJet's dominant airport is also Newark. Both airlines are situated on the far left of the x-axis, meaning they both face the same difficult airport-hour circumstances. However, United manages to overperform despite the difficult Newark conditions, being above the line of equality and ExpressJet falls dramatically below the line. Since we know they are flying out of the same dominant airport, the difference suggests that ExpressJet's bottom-tier punctuality ranking is a result of genuine operational inefficiency. Their poor punctuality cannot be fully blamed on the airport or schedule and that it is an internal performance issue.

The raw on-time departure ranking reveals the absolute punctuality of each airline, showing passengers exactly who departs on schedule most frequently.
However, it completely obscures the difficulty of the environments they fly in, unfairly penalizing airlines that operate heavily out of delay-prone airports.
By accounting for these airport-hour circumstances, we can separate operational efficiency from schedule disadvantages, revealing that a middle-ranked airline like United is actually overperforming its difficult Newark schedule, while a bottom-ranked carrier like ExpressJet genuinely underperforms despite facing the same constraints.

Are we on the right track?

Your team's answer is excellent — it is one of the strongest analytical responses you could write for this question. Here is my assessment.

### What Your Team Nailed ✅

**The United analysis is very well done.** Citing the specific numbers (~75.8% expected vs 78.2% observed), correctly identifying the position above the line of equality, and connecting it back to Newark as the dominant airport is exactly the level of precision the question asks for.[300]

**The Envoy comparison is a smart bonus insight** that goes beyond what the question strictly asks. Noting that Envoy falls *below* the line despite being 5th in raw ranking, while United is above it, demonstrates genuine critical reading of the plot — this kind of observation would likely impress a marker.

**The ExpressJet comparison is the strongest part.** Identifying that both United and ExpressJet share nearly identical expected on-time rates (~75–76%) and the same dominant airport, yet diverge dramatically in observed performance, is the exact inferential leap the lab has been building toward across all sections.

**The concluding paragraph is particularly strong.** The three-sentence synthesis — raw ranking reveals absolute punctuality but obscures difficulty; accounting for airport-hour circumstances separates efficiency from disadvantage; United overperforms while ExpressJet genuinely underperforms — is a clear, well-structured argument that directly answers the spirit of the entire lab section.

### Minor Suggestions 

**One small logical refinement on the "should be ranked 5th" claim.** Saying United "should be ranked 5th instead of 6th" based on the equality line is a reasonable intuition, but slightly informal. A more precise framing would be:

> *"On a schedule-adjusted basis, United's outperformance relative to its baseline suggests its true operational rank is higher than 6th — potentially above airlines like Envoy that fall below the line despite better raw numbers."*

This avoids implying a definitive re-ranking, which would require a formal adjusted score.

**Consider naming the key concept.** What your team has described is called **schedule-adjusted performance** or **expected vs. observed analysis** — essentially a form of controlling for confounds. Naming it explicitly would show theoretical awareness:

> *"This adjusted comparison separates genuine operational efficiency from schedule disadvantage..."*

You already say something close to this — just make it slightly more explicit.

### Overall Verdict

This is a well-argued, evidence-grounded, and analytically mature answer. It correctly reads the plot, draws the right comparisons, and arrives at a defensible conclusion that the lab was designed to lead you toward. Well done to your team.

⁂

1. https://opensource.posit.co/software/nycflights13/
2. https://nycflights13.tidyverse.org/reference/flights.html
3. https://glasgow-stats-l1-l2.github.io/DAS_Lab1/index.html
4. https://www.oreilly.com/library/view/hands-on-data-science/9781789139402/8b666bb6-64ed-4fc7-8bae-b28110ecc683.xhtml
5. https://subscription.packtpub.com/book/data_science/9781789139402/3/ch03lvl1sec18/tutorial-looking-at-airline-flight-times-data
6. https://www.rdocumentation.org/packages/dplyr/versions/0.5.0/topics/nycflights13
7. https://cran.r-project.org/package=nycflights13
8. http://vaibhavwalvekar.github.io/Portfolio_NYCFlights.pdf
9. https://stackoverflow.com/questions/76181882/problem-with-dataset-dbplyr-nycflights13
10. https://rpubs.com/cartooon/nycflights13
11. https://github.com/tidyverse/nycflights13
12. https://cran.r-project.org/web/packages/nycflights13/nycflights13.pdf
13. https://stackoverflow.com/questions/76019319/how-to-download-nycflights-13-package
14. https://stackoverflow.com/questions/74114085/visualizing-relational-database-such-as-nycflights13-in-r
15. https://forum.posit.co/t/nycflights13-package-is-not-found/15999
16. https://dplyr.tidyverse.org/reference/recode-and-replace-values.html
17. https://dplyr.tidyverse.org/reference/recode.html
18. https://sparkbyexamples.com/r-programming/replace-using-dplyr-package-in-r/
19. https://www.statology.org/dplyr-replace-multiple-values/
20. https://stackoverflow.com/questions/30382908/r-dplyr-rename-variables-using-string-functions
21. https://r-statistics.co/dplyr-rename-in-R.html
22. https://www.datanovia.com/en/lessons/rename-data-frame-columns-in-r/
23. https://cran.r-project.org/web/packages/dplyr/vignettes/dplyr.html
24. https://www.geeksforgeeks.org/r-language/how-to-replace-multiple-values-in-data-frame-using-dplyr/
25. https://www.youtube.com/watch?v=nbYgez6QqIs
26. https://www.rdocumentation.org/packages/dplyr/versions/0.7.8/topics/recode
27. https://www.rdocumentation.org/packages/dplyr/versions/1.0.10/topics/recode
28. https://stackoverflow.com/questions/32483280/use-dplyr-to-replace-values-on-select-columns/32484832
29. https://stackoverflow.com/questions/59827756/recode-replace-variables-conditionally-with-r-dyplyr
30. https://dplyr.tidyverse.org/reference/na_if.html
31. https://www.r-bloggers.com/2022/06/how-to-recode-values-in-r/
32. https://github.com/tidyverse/dplyr/issues/425
33. https://tidyverse.r-universe.dev/articles/dplyr/recoding-replacing.html
34. https://cran.r-project.org/web/packages/dplyr/vignettes/recoding-replacing.html
35. https://dplyr.tidyverse.org/reference/rename.html

36. https://dplyr.tidyverse.org/articles/recoding-replacing.html
37. https://r-coder.com/rename-dplyr-r/
38. https://www.statology.org/dplyr-rename-column/
39. https://tavareshugo.github.io/r-intro-tidyverse-gapminder/04-manipulate_variables_dplyr/index.html
40. https://github.com/KeremTurgutlu/r4ds_answers/blob/master/Chapter 5.Rmd
41. https://lokhc.wordpress.com/r-for-data-science-solutions/chapter-5-data-transformation/
42. https://github.com/danielfrg/r4ds-solutions/blob/master/5-data-transformation.Rmd
43. https://gedeck.github.io/DS-6030/book/data-transformation.html
44. https://dept.stat.lsa.umich.edu/~jerrick/courses/stat506_f23/ps04solns.html
45. https://aditya-dahiya.github.io/RfDS2solutions/Chapter4.html
46. https://r4ds.hadley.nz/data-transform.html
47. https://www2.stat.duke.edu/courses/Summer16/sta101.001-2/post/labs/lab2.html
48. https://forcats.tidyverse.org/reference/fct_reorder.html
49. https://r-graph-gallery.com/267-reorder-a-variable-in-ggplot2.html
50. https://forcats.tidyverse.org
51. https://www.youtube.com/watch?v=WRirYQBPq9s
52. https://www.rdocumentation.org/packages/forcats/versions/0.2.0/topics/fct_reorder
53. https://www.geeksforgeeks.org/r-language/how-to-order-boxes-in-boxplot-with-fct_reorder-in-r/
54. https://stackoverflow.com/questions/48799074/how-to-reorder-a-factor-in-a-dataframe-with-fct-reorder
55. https://stackoverflow.com/questions/51378506/difference-of-fct-reorder-and-reorder
56. https://rstudio.github.io/cheatsheets/html/factors.html
57. https://r-statistics.co/forcats-fct_reorder-in-R.html
58. https://kelrenmor.github.io/STA101-002/labs/lab2/lab2
59. https://www.stephane-v-boucheron.fr/labs/scidon/_handout/lab-tables.html
60. https://pages.pomona.edu/~jsh04747/courses/math58/lab1_m58_s20_intro_to_data.pdf
61. https://gbotosh.github.io/r_4_data_science_solutions/
62. https://lionel-hutz.quarto.pub/r4ds-2e-solutions/ch_13_solutions.html
63. https://wilkelab.org/classes/SDS348/2019_spring/labs/lab4-solution.html
64. https://nustat.github.io/intro-stat-data-sci/0c-solutions-learning-checks.html
65. https://gribblelab.org/2812/slides/slides02.html
66. https://mine-cetinkaya-rundel.github.io/r4ds-solutions/data-transform.html
67. https://data.europa.eu/apps/data-visualisation-guide/axes-grids-and-legends
68. https://r-graph-gallery.com/267-reorder-a-variable-in-ggplot2.html
69. https://www.addtwodigital.com/add-two-blog/2021/8/18/rule-24-label-your-bars-and-axes
70. https://www.storytellingwithdata.com/blog/2017/8/2/axis-vs-data-labels
71. https://www.reddit.com/r/math/comments/2axe11/eli5_why_is_it_always_y_axis_vs_x_axis_for_the/
72. https://www.ventanasystems.co.uk/forum/viewtopic.php?t=8341

73. https://www.facebook.com/groups/203376080793152/posts/515306686266755/
74. https://www.forbes.com/sites/naomirobbins/2013/02/12/how-to-position-y-axis-labels-in-graphs/
75. https://policyviz.com/2016/09/07/where-to-position-the-y-axis-label/
76. https://statmodeling.stat.columbia.edu/2023/01/26/labeling-the-x-and-y-axes-heres-a-quick-example-where-a-bit-of-care-can-make-a-big-difference/
77. https://stackoverflow.com/questions/26826865/how-to-drop-unused-levels-after-filtering-by-factor
78. https://stackoverflow.com/questions/32059343/when-filtering-with-dplyr-in-r-why-do-filtered-out-levels-of-a-variable-remain
79. https://forcats.tidyverse.org/reference/fct_drop.html
80. https://www.r-bloggers.com/2022/06/droplevels-in-r-with-examples/
81. https://transtats.bts.gov/TARMAC/tarmacAirportData?6B2r=OB+Nv4yv0r&56zz=nyy&rqzz=nyy&Brn4=FDFE&6zv0=ELE&6znA=EDDDD
82. https://www.bts.gov/content/us-air-carrier-aircraft-departures-enplaned-revenue-passengers-and-enplaned-revenue-tons
83. https://www.transtats.bts.gov
84. https://krlmlr.github.io/vistransrep/2019-04-zhr/summarize.html
85. http://libraryonline.erau.edu/online-full-text/books-online/AAS/2003/aas_tbl1_2003.pdf
86. https://github.com/tidyverse/tidyr/issues/1508
87. https://www.rdocumentation.org/packages/REDCapCAST/versions/26.1.1/topics/fct_drop
88. https://rdrr.io/r/base/droplevels.html
89. https://www.statology.org/dplyr-filter-on-factor/
90. https://r-statistics.co/forcats-fct_expand-in-R.html
91. https://www.r-bloggers.com/2021/03/dropping-levels-in-a-factor-variable/
92. https://stat.ethz.ch/R-manual/R-devel/library/base/help/droplevels.html
93. https://scales.arabpsychology.com/stats/how-do-i-filter-dplyr-based-on-a-factor/
94. https://epirhandbook.com/en/new_pages/factors.html
95. https://www.reddit.com/r/Rlanguage/comments/1ih2nzy/r_isnt_finding_one_of_my_variables_when_i_try_to/
96. https://stat545.com/factors-boss.html
97. https://www.transtats.bts.gov/ontime/departures.aspx
98. https://www2.stat.duke.edu/courses/Spring18/Sta199/exam/midterm-01-post.html
99. https://assets.ctfassets.net/qiecpoxp4bka/74xIhACRC0xmMFSkUrjU7E/612886a15bb0e975c9aac41d8ef670e1/202512_Average_Daily_Departures.pdf
100. https://id.scribd.com/document/585315068/Introduction-to-data-2
101. https://www.oag.com/nam-on-time-performance-data
102. https://www.youtube.com/watch?v=Zlw1gZ45CFE
103. https://www.linkedin.com/posts/morgandepenbusch_there-is-one-data-viz-rule-that-cannot-activity-7325874744694624256-xoZI
104. https://academy.datawrapper.de/article/326-why-our-column-and-bar-charts-start-at-zero

105. https://digitalblog.ons.gov.uk/2016/07/23/does-the-axis-have-to-start-at-zero-part-2-bar-charts/
106. https://www.datawrapper.de/academy/why-our-column-and-bar-charts-start-at-zero
107. https://stephanieevergreen.com/y-axis/
108. https://www.reddit.com/r/unpopularopinion/comments/1gmu2dz/graphs_with_an_xaxis_that_doesnt_start_at_zero/
109. https://medium.com/@addtwo/rule-25-always-start-your-bar-charts-at-zero-11d7994ccd5b
110. https://www.storytellingwithdata.com/blog/2012/09/bar-charts-must-have-zero-baseline
111. https://www.datylon.com/blog/dot-plot-deep-dive
112. http://www.chadskelton.com/2018/06/bar-charts-should-always-start-at-zero.html
113. https://www.linkedin.com/pulse/should-axis-your-graph-always-start-zero-most-time-yes-dave-paradi
114. https://www.reddit.com/r/excel/comments/prvbhw/how_can_i_fix_the_xaxis_so_that_my_first_data/
115. https://stephanieevergreen.com/non-zero-axis-rules/
116. https://digitalblog.ons.gov.uk/2016/06/27/does-the-axis-have-to-start-at-zero-part-1-line-charts/
117. https://www.cnn.com/2026/01/05/travel/cirium-on-time-performance-review-2025
118. https://www.theflightclub.it/en/2026/01/2025-on-time-companies-rankings/
119. https://www.oag.com/hubfs/Free_Reports/Punctuality_League/2018/PunctualityReport2018.pdf
120. https://www.oag.com/on-time-performance-data
121. https://www.oag.com/hubfs/free-reports/2023/OAG-Punctuality-League-2023.pdf
122. https://traveltomorrow.com/the-worlds-most-punctual-airlines-and-airports-in-2025/
123. https://www.youtube.com/watch?v=jzSm86VrZdE
124. https://www.gdg.travel/blog/punctuality-league-2023/
125. https://cdn2.hubspot.net/hubfs/490937/free-reports/2016-reports/2016-punctuality-league/PunctualityReport2016.pdf
126. https://aws-a.medias-ccifi.org/fileadmin/cru-1718890700/template/japon/img/news/2016/01-janvier/OAG-punctuality/PunctualityReport2015_Final.pdf
127. https://www.bts.gov/topics/airline-time-tables
128. https://www.nj.com/news/2017/02/leaving_from_newark-liberty_chances_are_the_flight.html
129. https://www.transtats.bts.gov/ontime/
130. https://www.oag.com/nam-on-time-performance-data
131. https://usafacts.org/articles/what-are-the-best-and-worst-airlines-for-on-time-performance/
132. https://www.oag.com/hubfs/Free_Reports/Punctuality_League/2018/PunctualityReport2018.pdf
133. https://www.cbsnews.com/newyork/news/jfk-laguardia-newark-most-delayed-airports/
134. https://www.schumer.senate.gov/newsroom/press-releases/new-schumer-study-reveals-for-first-time-laguardia-jfk-and-newark-rank-as-worst-three-in-nation-for-delayed-flights
135. https://www.oag.com/hubfs/free-reports/2023/OAG-Punctuality-League-2023.pdf
136. https://6abc.com/archive/7132457/
137. https://krlmlr.github.io/vistransrep/2018-11-zhr/dplyr-2.html
138. https://www.stephane-v-boucheron.fr/labs/scidon/_handout/lab-tables.html
139. https://rstudioconnect.wlu.edu/bio185/_w_d2f70aeb/c-07-exploring-data.html

140. https://ui-research.github.io/r4ds-exercises/chapter-5-data-transformation.html
141. https://www2.stat.duke.edu/courses/Summer16/sta101.001-2/post/labs/lab2.html
142. https://hranalyticslive.netlify.app/d-appendixd
143. https://jrnold.github.io/r4ds-exercise-solutions/transform.html
144. https://lionel-hutz.quarto.pub/r4ds-2e-solutions/ch_13_solutions.html
145. https://erilu.github.io/R-for-data-science-walkthrough/chapter-13-relational-data.html
146. https://writingcenter.uci.edu/2023/08/18/summarizing/
147. https://www.grammarly.com/ai/ai-writing-tools/summarizing-tool
148. https://www.youtube.com/watch?v=YZ3_FgnC4Cg
149. https://www.lib.sfu.ca/about/branches-depts/slc/writing/sources/summarizing
150. https://www.youtube.com/watch?v=Z1vqOFTvQkk
151. https://www.readingrockets.org/classroom/classroom-strategies/summarizing
152. https://writingcenter.uconn.edu/wp-content/uploads/sites/593/2014/06/How_to_Summarize_a_Research_Article1.pdf
153. https://study.com/learn/lesson/what-is-a-summary.html
154. https://www.mindtools.com/axggxkv/paraphrasing-and-summarizing/
155. https://www.soka.edu/writing-center/writing-summaries
156. https://www.bts.gov/topics/airline-time-tables
157. https://www.transportation.gov/sites/dot.gov/files/2025-03/February 2024 ATCR revised 3-06-2025.pdf
158. https://www.oag.com/hubfs/Free_Reports/Punctuality_League/2018/PunctualityReport2018.pdf
159. https://www.transportation.gov/sites/dot.gov/files/2020-11/November 2020 ATCR.pdf
160. https://www.oag.com/hubfs/free-reports/2023/OAG-Punctuality-League-2023.pdf
161. https://www.oag.com/nam-on-time-performance-data
162. https://www.bitre.gov.au/statistics/aviation/otp_month
163. https://www.transtats.bts.gov/ontime/
164. https://www.bts.gov/annual-time-departure-performance
165. https://business.columbia.edu/sites/default/files-efs/pubfiles/546/546.pdf
166. image-2.jpg
167. image.jpg
168. https://tidyr.tidyverse.org/reference/complete.html
169. https://www.youtube.com/watch?v=tCpGvCHLHEA
170. https://r4ds.hadley.nz/missing-values.html
171. https://www.internetgeography.net/calculate-a-percentage/
172. https://dev.to/jfcaba/how-i-built-a-delay-risk-score-for-628000-flights-1o5e
173. https://www.transtats.bts.gov/ontime/
174. https://gmatclub.com/forum/for-each-of-5-airlines-airlines-1-through-5-the-table-shows-the-per-418146.html
175. https://search.r-project.org/CRAN/refmans/tidyr/html/complete.html

176. https://stackoverflow.com/questions/46685510/fill-missing-combinations-in-a-dataframe
177. https://tidyr.tidyverse.org/reference/expand.html
178. https://github.com/tidyverse/tidyr/issues/127
179. https://rdrr.io/cran/tidyr/man/expand.html
180. https://tidyr.tidyverse.org/reference/fill.html
181. https://www.rdocumentation.org/packages/tidyr/versions/0.4.1/topics/complete
182. https://luisdva.github.io/rstats/complete/
183. https://dabblingwithdata.amedcalf.com/2021/06/26/create-rows-for-missing-combinations-of-data-with-r/
184. https://stackoverflow.com/questions/42866119/fill-missing-values-in-data-frame-using-dplyr-complete-within-groups
185. https://r-statistics.co/tidyr-complete-in-R.html
186. https://github.com/tidyverse/tidyr/issues/396
187. https://www.calculatorsoup.com/calculators/math/percentage.php
188. https://byjus.com/maths/how-to-calculate-percentage/
189. https://www.linkedin.com/posts/saiyyed-murtaza-ali-naqvi-9016a436_sql-dataanalytics-databaseperformance-activity-7295761528333422594-jOY3
190. https://www.icao.int/sites/default/files/sp-files/SAM/Documents/2020-RLA06901-ANPVOLIII/KPI 1 al 19 detallados.pdf
191. https://www.aviationdb.com/Aviation/F4SDetailQuery.shtm
192. https://support.microsoft.com/en-us/office/calculate-percentages-6b5506e9-125a-4aba-a638-d6b40e603981
193. https://tidyr.tidyverse.org/reference/complete.html
194. https://www.statswithr.com/r-functions/the-complete-function-in-r
195. https://www.youtube.com/watch?v=tCpGvCHLHEA
196. https://r-statistics.co/tidyr-complete-in-R.html
197. https://stackoverflow.com/questions/74547518/how-to-use-the-tidyr-complete-and-fill-functions-to-extend-a-column-of-nas-in-r
198. https://luisdva.github.io/rstats/complete/
199. https://sparkbyexamples.com/data-engineering/explain-complete-function-in-r-with-examples/
200. https://www.rdocumentation.org/packages/tidyr/versions/0.4.1/topics/complete
201. https://tidyr.tidyverse.org
202. https://www.youtube.com/watch?v=whQOPDoqPl0
203. https://cran.r-project.org/web/packages/tidyr/refman/tidyr.html
204. https://r-graph-gallery.com/79-levelplot-with-ggplot2.html
205. https://ggplot2.tidyverse.org/reference/coord_flip.html
206. https://jcoliver.github.io/learn-r/006-heatmaps.html
207. https://cran.r-project.org/web/packages/ggalign/vignettes/special-syntax.html
208. https://bioinformatics.ccr.cancer.gov/docs/data-visualization-with-r/Lesson5_intro_to_ggplot_version5/
209. https://ggplot2.tidyverse.org/articles/faq-axes.html
210. https://stackoverflow.com/questions/8406394/how-to-produce-a-heatmap-with-ggplot2

211. https://rud.is/projects/facetedheatmaps.html
212. https://www.geeksforgeeks.org/r-language/create-heatmap-in-r-using-ggplot2/
213. https://stackoverflow.com/questions/66861754/preparing-data-frame-in-r-for-heatmap-with-ggplot2
214. https://epirhandbook.com/en/new_pages/heatmaps.html
215. https://www.youtube.com/watch?v=5oUg0TLDe6I
216. https://www.oreilly.com/library/view/r-graphics-cookbook/9781449363086/ch08.html
217. https://r-statistics.co/Heatmap-in-R.html
218. https://www.youtube.com/watch?v=-4sGamyqGAQ
219. https://r-graph-gallery.com/79-levelplot-with-ggplot2.html
220. image.jpg
221. https://usafacts.org/articles/what-are-the-best-and-worst-airlines-for-on-time-performance/
222. image.jpg
223. https://www.cirium.com/thoughtcloud/the-increasing-significance-of-timely-performance/
224. https://www.reddit.com/r/unitedairlines/comments/1li228c/abysmal_ontime_performance/
225. https://www.allthingsontimeperformance.com/punctuality-versus-on-time-performance/
226. https://www.flightstats.com/v2/
227. https://www.cirium.com/resources/on-time-performance/
228. https://www.instagram.com/reel/DU_AA8GFCPc/?hl=en
229. https://www.oag.com/on-time-performance-data
230. https://www.transportation.gov/airconsumer/fly-rights
231. https://www.oag.com/airline-on-time-performance-defining-late
232. https://www.sciencedirect.com/science/article/pii/S2590198221000932
233. https://www.facebook.com/PeterGreenbergWorldwide/posts/many-airlines-routinely-pad-their-schedules-anywhere-from-20-to-40-minutes-more-/868857448136759/
234. https://www.bts.gov/topics/airline-time-tables
235. https://www.instagram.com/p/DTrMNGbkcXk/?hl=en
236. https://www.bitre.gov.au/statistics/aviation/otp_month
237. https://backroadplanet.com/the-best-time-of-day-to-fly-if-you-want-fewer-delays-according-to-experts/
238. https://thepointsguy.com/news/whats-the-best-time-to-fly-we-analyzed-7-million-flights-to-find-out/
239. image.jpg
240. image.jpg
241. https://www.booking.com/guides/article/flights/the-best-days-to-fly-for-fewer-delays-and-cancellations.html
242. https://www.forbes.com/2008/10/14/travel-airports-time-forbeslife-cx_fl_1014travel.html
243. https://www.businesstimes.com.sg/companies-markets/transport-logistics/changi-airport-ranked-fourth-globally-time-performance
244. https://aviation.direct/en/Why-the-departure-time-determines-punctuality
245. https://www.reddit.com/r/dataisbeautiful/comments/843hnj/airline_ontime_performance_on_a_24hour_clock_oc/

246. https://zipdo.co/flight-ontime-statistics/
247. https://sg.trip.com/ask/questions/best-day-to-book-flights.html
248. https://lessonsfromtheflightdeck.substack.com/p/the-flight-times-everyone-hates-and
249. https://www.oag.com/on-time-performance-data
250. https://www.bts.gov/topics/airline-time-tables
251. https://www.linkedin.com/posts/safar-on-time_timing-plays-a-crucial-role-in-avoiding-flight-activity-7244669542461804546-8Bnp
252. https://www.frommers.com/tips/airfare/5-reasons-early-morning-flights-are-less-likely-to-be-delayed/
253. image.jpg
254. https://backroadplanet.com/the-best-time-of-day-to-fly-if-you-want-fewer-delays-according-to-experts/
255. https://mason.gmu.edu/~wcao2/Milestone2.pdf
256. http://bigdatasummerinst.sph.umich.edu/wiki2019/images/6/63/Bdsi_2019_r_practice_dplyr_nycflights_answers.pdf
257. https://www2.stat.duke.edu/courses/Spring18/Sta199/exam/midterm-01-post.html
258. http://snap.stanford.edu/class/cs224w-2013/projects2013/cs224w-056-final.pdf
259. https://soutik.github.io/NYC-Flight-Analysis/
260. https://traffic-viz.github.io/gallery/heatmaps.html
261. https://id.scribd.com/document/585315068/Introduction-to-data-2
262. https://www.scribd.com/document/915465900/LAB-06
263. https://openflights.org
264. https://www.coursehero.com/file/213453245/Assignment-3pdf/
265. https://www2.stat.duke.edu/courses/Summer16/sta101.001-2/post/labs/lab2.html
266. https://aditya-dahiya.github.io/RfDS2solutions/Chapter14.html
267. https://www.caa.co.uk/Documents/Download/2101/1767f649-35a4-40dc-9ed2-ef665805e270/58
268. https://stephane-v-boucheron.fr/labs/eda/_handout/lab-tables.pdf
269. https://krlmlr.github.io/vistransrep/2018-11-zhr/dplyr-2.html
270. https://moderndive.com/D-appendixD.html
271. https://pt.scribd.com/document/585315031/Week-2-Lab-Introduction-to-Data-Coursera
272. https://www.caa.co.uk/Documents/Download/2106/913c5c9b-7f20-4d20-9225-2bea79497cc9/29
273. https://rdrr.io/cran/lvplot/man/ontime.html
274. https://r-statistics.co/Bubble-Chart-in-R.html
275. https://r-charts.com/correlation/bubble-chart-ggplot2/
276. https://r-graph-gallery.com/package/ggrepel.html
277. https://r-graph-gallery.com/web-scatterplot-and-ggrepel.html
278. https://jmsallan.netlify.app/blog/dense-visualization-in-r-with-ggrepel-and-plotly/
279. https://r-graph-gallery.com/320-the-basis-of-bubble-plot.html
280. https://r-statistics.co/ggplot2-geom_text-in-R.html
281. https://www.youtube.com/watch?v=IeFVs37HrA8

282. https://r-graph-gallery.com/330-bubble-map-with-ggplot2.html
283. https://github.com/slowkow/ggrepel/blob/master/vignettes/examples.Rmd
284. https://www.youtube.com/watch?v=9wEW8DIIo_k
285. https://github.com/kyle-w-brown/R-Gallery/blob/main/correlation.md
286. https://ggrepel.slowkow.com/articles/examples
287. https://stackoverflow.com/questions/53430041/repel-geom-label-and-text-in-ggplot-and-ordering-geom-points-based-on-size
288. https://stackoverflow.com/questions/53392133/increase-bubble-size-in-geom-point-not-label-using-ggplot2-gganimate-ggrep
289. https://ggplot2.tidyverse.org/reference/geom_abline.html
290. https://plotnine.org/reference/geom_abline.html
291. https://stackoverflow.com/questions/39197217/using-geom-abline-and-ggplot
292. https://www.sthda.com/english/wiki/ggplot2-add-straight-lines-to-a-plot-horizontal-vertical-and-regression-lines
293. https://www.rdocumentation.org/packages/ggplot2/versions/0.9.1/topics/geom_abline
294. https://forum.posit.co/t/add-x-y-geom-abline-to-ggplot/32154
295. https://tidyverse.github.io/ggplot2-docs/reference/geom_abline.html
296. https://github.com/yhat/ggpy/issues/309
297. https://lets-plot.org/python/pages/api/lets_plot.geom_abline.html
298. https://groups.google.com/g/ggplot2/c/_HeG-CYBCGc
299. image.jpg
300. image.jpg